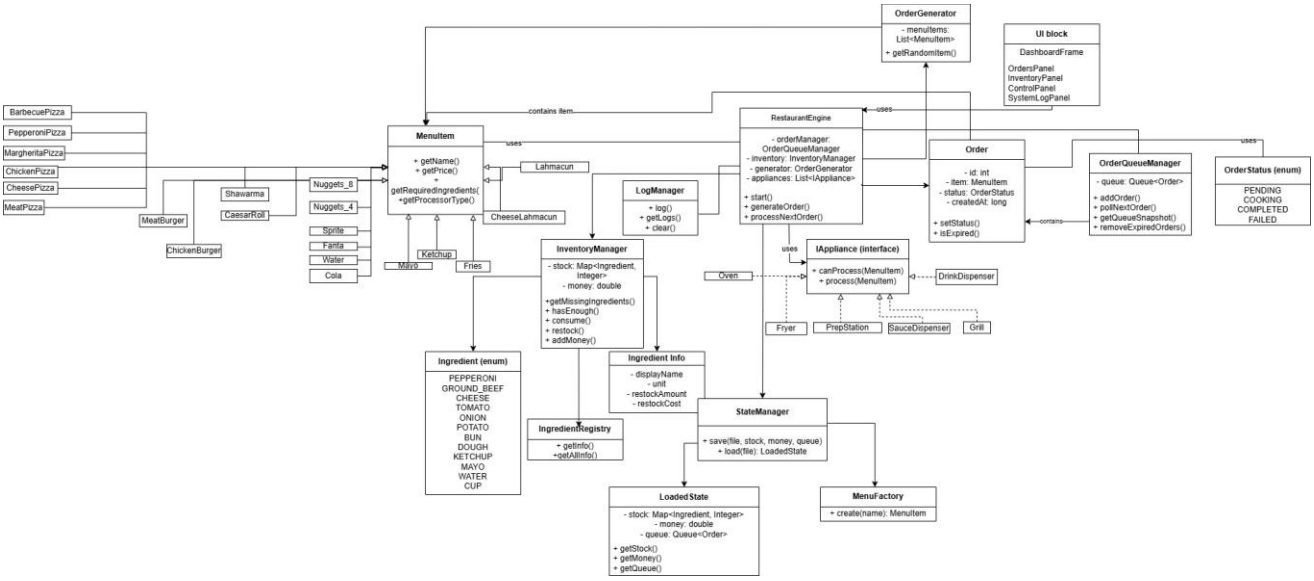
3.1 System Overview

The Silicon Spatula is a restaurant management simulation built in Java Swing. The system automatically generates orders using an internal Swing Timer and places them into a First In First Out order queue managed by OrderQueueManager. Each order is a instance of a MenuItem, and menu item classes such as pizzas, burgers, fries, drinks, sauces, lahmacun, and nuggets define their own prices, needed ingredients, and processor type. When the user presses Cook Next Order button, RestaurantEngine checks the next order, checks the required ingredients through InventoryManager, and only removes the order from the queue if the resources are enough to cook the order. The matching kitchen appliance is choosen polymorphically by iterating through a list of IAppliance objects and using canProcess(item) method. If successful, the required ingredients are used, the appliance processes the item, the order status is updated, and money is added to the restaurant balance. On failure, the system logs an error and avoids changing the inventory, money, or queue state. The Swing user interface contains separate panels for the order queue, inventory/resources, restocking, control buttons, and system log, and these panels refresh after generation, processing, restocking, saving, and loading. The StateManager class supports saving and loading the current money, ingredient stock, pending queue order, order status, order ID, creation time, and menu item type so the simulation can continue from a saved state.

3.2 System Overview

The Silicon Spatula is a restaurant management simulation built in Java Swing. The system automatically generates orders using an internal Swing Timer and places them into a First In First Out order queue managed by OrderQueueManager. Each order is a instance of a MenuItem, and menu item classes such as pizzas, burgers, fries, drinks, sauces, lahmacun, and nuggets define their own prices, needed ingredients, and processor type. When the user presses Cook Next Order button, RestaurantEngine checks the next order, checks the required ingredients through InventoryManager, and only removes the order from the queue if the resources are enough to cook the order. The matching kitchen appliance is choosen polymorphically by iterating through a list of IAppliance objects and using canProcess(item) method. If successful, the required ingredients are used, the appliance processes the item, the order status is updated, and money is added to the restaurant balance. On failure, the system logs an error and avoids changing the inventory, money, or queue state. The Swing user interface contains separate panels for the order queue, inventory/resources, restocking, control buttons, and system log, and these panels refresh after generation, processing, restocking, saving, and loading. The StateManager class supports saving and loading the current money, ingredient stock, pending queue order, order status, order ID, creation time, and menu item type so the simulation can continue from a saved state.

3.3 UML Class Diagram



3.4 Key Design Decisions

How Tasks Are Structured

Orders in the system are represented by the Order class, and each Order is a MenuItem. MenuItem is an abstract class with attributes name and price, plus the abstract methods getRequiredIngredients() and getProcessorType(). Classes that extend MenuItem are CheesePizza, PepperoniPizza, ChickenBurger, MeatBurger, Shawarma, CaesarRoll, Fries, Nuggets_4, Nuggets_8, Lahmacun, CheeseLahmacun, Cola, Sprite, Fanta, Water, Ketchup, and Mayo define their own required ingredients and processor type. This keeps the hierarchy simple while still using inheritance, because each menu item has different behavior through overridden methods. New menu items can be added by creating another class which extends MenuItem abstract class without changing the engine logic.

How Processors Are Selected

We set up a system where menu items are handled using an IAppliance interface. This interface comes with two main tasks: canProcess() and process(). Every piece of equipment in our system, like an oven or a fryer, follows these two functions.
We made different kinds of appliance classes – like an Oven, a Fryer, a Grill, a Drink Dispenser, a Prep Station, and a Sauce Dispenser. Each of these appliances knows if it can handle a specific menu item by checking its 'processor type'. To make this work, every menu item has a getProcessorType() function that tells us what kind of processing it needs. The appliance then simply compares that with its own type.
So, when someone places an order, our main system, the RestaurantEngine, goes through each appliance one by one. It asks each one, "Can you process this?" using canProcess() function. The very first appliance that says "yes" then gets the job of processing that item.
We first thought about just using a lot of 'if-else' statements or checking each item type directly. But we quickly realized that would make the code much longer and a real pain to manage. So, instead, we decided to let each appliance figure it out on its own. This keeps the main engine simple; it just acts as a connector for everything. Plus, if we ever want to add something new later, like a new type of appliance, we won't need to change much of the code that's already there.

How Resources Are Validated and Consumed

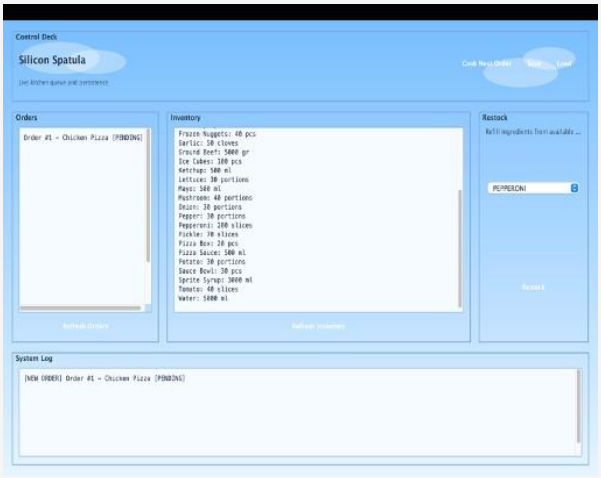
InventoryManager is responsible for all resource validation and usage. Ingredient quantities are stored in a private HashMap<Ingredient, Integer>, so the GUI and engine cannot directly modify the internal stock map. Before an order is cooked, RestaurantEngine takes the required ingredients from the MenuItem and uses hasEnough() method to verify that every needed ingredient is in stock. If ingredients are not enough to cook the certain MenuItem, the order is considered as failed, an error is displayed to LogManager, stock and money values are not changed. If resources are enough, the order is removed from the queue and consume() method deducts the required ingredient amounts. The consume() method checks the stock again and prevents negative values. Restocking is also handled through InventoryManager, using IngredientRegistry and IngredientInfo to determine each ingredient's restock amount, unit, and cost.

How Save / Load Works

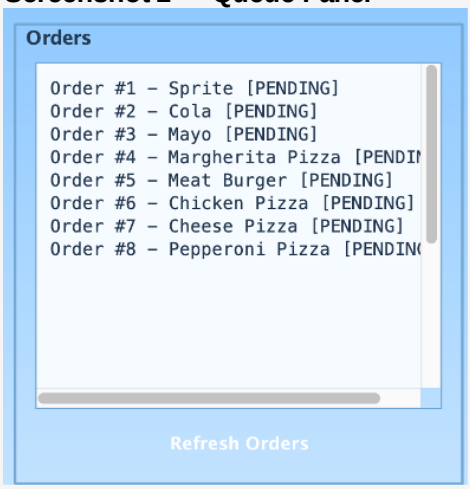
We set up a system to save and load everything in our project using a class called StateManager. The main idea was to save the current state of the system into a file so you can always pick up where you left off without losing any progress.
When you hit save, the system essentially writes all the current information into a text file, like "save.txt". It starts by recording how much money you have. After that, it lists out all the ingredients in your inventory and their quantities. Then, it goes through your order queue, saving each order one by one. This includes details like what the item is called, its current status, and other information it would need to put the order back together later.
For loading, the system simply reads that same file, line by line. It first puts your money value back to what it was. Then, it rebuilds your inventory by reading all those ingredient quantities. After that, it recreates your order queue, putting everything back in the exact order it was saved.
To make sure the menu items reappear correctly, we used a pretty straightforward "factory" method. It just takes the item's name and uses that to create the right MenuItem object again. This way, every order gets restored with the correct type and all its data intact.
We also added a quick check to make sure the save file actually exists before trying to load anything. This is a nice little safeguard, so the program doesn't crash if someone tries to load something when they haven't saved anything yet.
All in all, this save and load feature really makes the system much more practical. Users can stop anytime and then continue their progress later without having to restart everything from the very beginning.

3.5 Screenshots

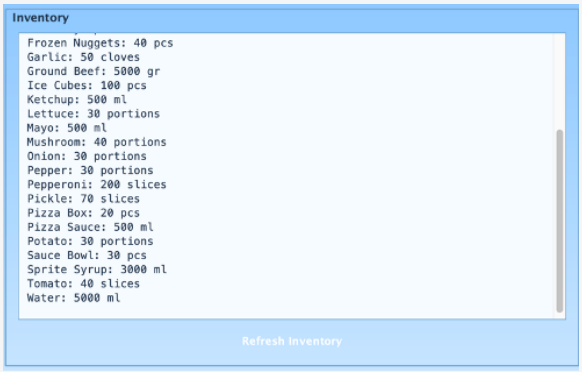
Screenshot 1 — Main UI



Screenshot 2 — Queue Panel



Screenshot 3 — Resource Panel



Screenshot 4 — System Log